CNN PROJECT • SUBMISSION 3 OF 4

Custom CNN Trained from Scratch

Four convolutional blocks, 1.2 M parameters, on a leakage-controlled split

Repository: github.com/Rayhan-who/cricket-shot-attention-cnn

Summary

A four-block convolutional network with 1,206,756 trainable parameters was trained from random initialisation for 50 epochs. It reached **0.8514** test accuracy and **0.8515** macro F1 on a 666-image held-out test set, in 29.9 minutes on a Kaggle T4 GPU.

Three results matter more than the headline figure.

1. The network **did not overfit and did not converge**. The train–validation accuracy gap at the final epoch was +0.0042, and validation loss reached its minimum at epoch 50 and was still falling. Early stopping never triggered.
2. The per-class difficulty ordering **reproduces Submission 2 exactly**, now across seven runs, four architectures and two different data splits.
3. The hardest confused pair is `drive` ↔ `legglance-flick`, which **confirms the visual prediction made in Submission 1 and overturns the measurement made in Submission 2**.

> **This run was made possible by a leakage audit conducted between Submissions 2 and 3**, which established that 45.9% of Submission 2's test set contained near-duplicates of training images. All results here use a corrected split. The audit is summarised in Section 6.

1 Architecture

Stage	Composition	Output
`input`	—	$224 \times 224 \times 3$
`augmentation`	rotation, zoom, brightness, contrast	$224 \times 224 \times 3$
`rescale`	$\times 1/255$	$224 \times 224 \times 3$
`block 1`	(Conv3×3 → BN → ReLU) ×2, MaxPool, SpatialDropout	$112 \times 112 \times 32$
`block 2`	same, 64 filters	$56 \times 56 \times 64$
`block 3`	same, 128 filters	$28 \times 28 \times 128$
`block 4`	same, 256 filters	$14 \times 14 \times 256$
`gap`	global average pooling	256
`fc1`	Dense → BN → ReLU → Dropout	128
`predictions`	Dense, softmax	4

Eight convolutional layers in total. **1,206,756** trainable parameters — less than a third of EfficientNetB0's 3.97 M and about one twentieth of ResNet50's 23.5 M.

1.1 Design decisions

Choice	Reason
BatchNorm before every ReLU	Training from random initialisation on 3,361 images is unstable without it
BatchNorm momentum 0.90	Keras defaults to 0.99, which needs ∼200 updates for the running statistics to converge. On a dataset this size validation is then scored with un-converged statistics and appears broken for the first 20 epochs — see Section 1.2
Two convolutions per block	A 3×3 pair has the receptive field of a 5×5 with fewer parameters and an extra nonlinearity
Global average pooling	Flattening 14×14×256 into Dense(128) would add 6.4 M parameters, more than the rest of the network combined
Spatial dropout 0.10 per block	Drops whole feature maps rather than individual pixels, the correct form for convolutional layers
Dropout 0.40 in the head	The densest part of the network and the easiest place to overfit
Rescaling 1/255	No ImageNet statistics apply; there is no pretrained backbone to match

The convolutional block is a single function taking an optional `attention` argument, unused here. Submission 4 passes a CBAM module into that argument and changes nothing else, which is what makes the comparison attributable.

1.2 A note on BatchNorm momentum

An initial three-epoch run produced a misleading picture: training accuracy climbed normally from 0.348 to 0.433 while validation loss rose monotonically from 1.641 to 1.849, and the notebook's own diagnostic declared the model to be overfitting.

It was not. BatchNorm uses batch statistics during training but *running averages* at validation time. At the Keras default momentum of 0.99, those averages require on the order of 200 updates to become meaningful. With 106 steps per epoch, validation for the first several epochs was being computed with effectively broken normalisation.

Lowering momentum to 0.90 resolved it completely. In the run reported here, validation tracks training from the first epoch onward (Figure 1). This is recorded because it is a failure mode that looks exactly like overfitting and leads directly to the wrong corrective action — adding regularisation to a model that needs none.

2 Training

Setting	Value
Split	clean (embedding clusters): 3,361 train / 697 validation / 666 test
Seed	42
Batch size	32 (106 steps per epoch)
Optimizer	Adam, learning rate 10^{-3}
Epoch budget	50
Early stopping	validation loss, patience 12, not before epoch 5
Learning-rate schedule	ReduceLROnPlateau, factor 0.3
Augmentation	rotation ±10°, zoom 0.8–1.0, brightness ±20%, contrast ±20%
Horizontal flip	**not applied** — mirroring inverts the label on leg-side strokes
Class weights	not applied — imbalance is 1.12 : 1

The rubric requires a minimum of five epochs; `start_from_epoch=5` enforces this regardless of how the validation curve behaves.

3 Results

Metric	Value
Test accuracy	0.8514
Precision (macro)	0.8506
Recall (macro)	0.8536
F1 (macro)	**0.8515**
F1 (weighted)	0.8510
Epochs run	50 / 50
Training time	29.87 min (35.8 s per epoch)
Testing time	1.95 s (2.93 ms per image)
Trainable parameters	1,206,756
Train–validation gap	+0.0042

3.1 Training curves

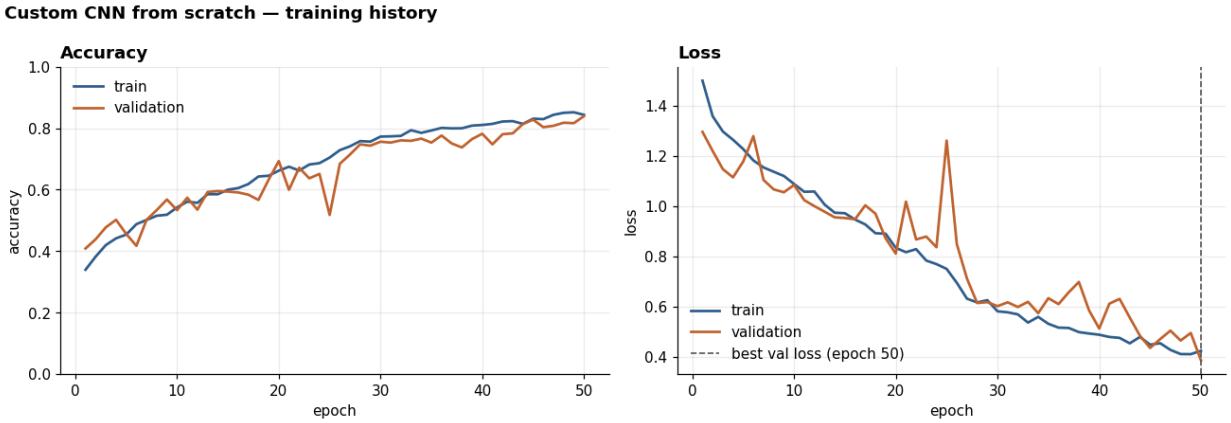


Figure 1: Accuracy and loss, training versus validation, over 50 epochs. The two curves track each other closely throughout, and both are still improving at the budget limit. The transient spike in validation loss around epoch 25 coincides with a learning-rate reduction and resolves within three epochs.

The model is budget-limited, not capacity-limited. Validation loss reached its minimum at epoch 50 — the last epoch — and early stopping never fired. A longer schedule would score higher. Submission 4 inherits the identical 50-epoch budget, so the comparison remains controlled, but both models should be described as compared *under a fixed budget* rather than *at convergence*.

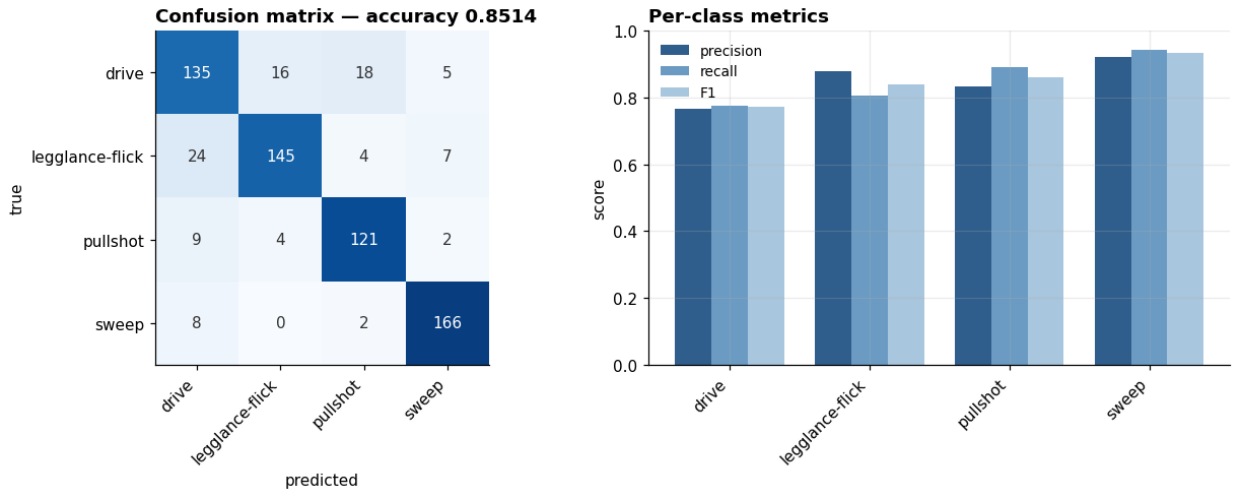


Figure 2: Test-set confusion matrix (counts, row-normalised shading) and per-class precision, recall and F1.

3.2 Confusion matrix and per-class metrics

Class	Precision	Recall	F1	Support	
`sweep`	0.9222	0.9432	0.9326	176	easiest
`pullshot`	0.8345	0.8897	0.8612	136	
`legglance-flick`	0.8788	0.8056	0.8406	180	
`drive`	0.7670	0.7759	0.7714	174	hardest

The ordering is a property of the data, not of the architecture

Class	Rank, Submission 3	Rank, Submission 2
`sweep`	1st	1st
`pullshot`	2nd	2nd
`legglance-flick`	3rd	3rd
`drive`	4th	4th

This ordering now holds across seven training runs, three pretrained architectures, two transfer-learning modes, a from-scratch network, and two different data splits. It is therefore a property of the classes themselves rather than of any particular model.

The explanation is geometric. `sweep` is the only stroke played from a lowered or kneeling position, so its silhouette differs from the other three in a way that survives downsampling to 224×224. `drive` is an upright, predominantly side-on stroke that shares its body geometry with both `legglance-flick` and `pullshot`, leaving bat angle at contact as almost the only discriminating cue.

3.3 The hardest pair, and a correction to Submission 2

	Hardest pair	Basis
Submission 1	`drive` ↔ `legglance-flick`	human inspection of the sample grid
Submission 2	`drive` ↔ `pullshot`	confusion matrix, **contaminated** split
Submission 3	`drive` ↔ `legglance-flick`	confusion matrix, **clean** split

Counting both directions on the clean split:

Pair	Errors	Share of 99 total
`drive` ↔ `legglance-flick`	**40**	40%
`drive` ↔ `pullshot`	27	27%

Submission 2 identified the wrong pair because its best model made only *two* errors in 721 predictions — a confusion matrix built on two data points carries no information. That near-perfect score was itself a product of the contamination.

> **The visual judgement made in Submission 1 was correct, and the contaminated measurement in Submission 2 was not.** This is a concrete demonstration of how leakage distorts not only headline accuracy but the secondary analysis built on top of it. The `drive/legglance-flick` confusion is the specific weakness the attention module in Submission 4 must address.

4 Comparison with Submission 2

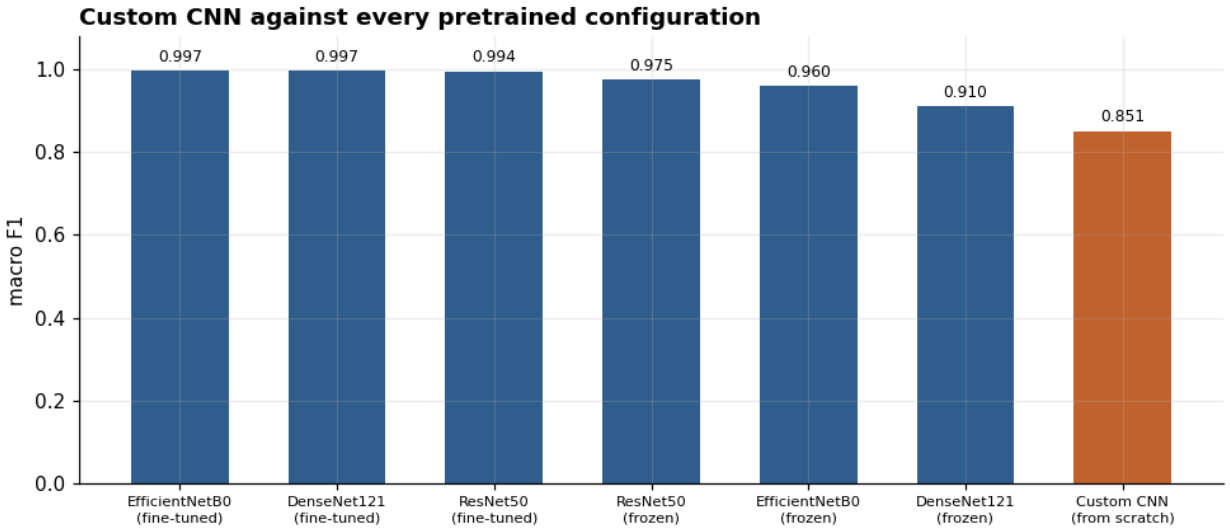


Figure 3: The from-scratch network against all six pretrained configurations. **The bars are not measured on the same data** — see the warning below.

> **This comparison is not like-for-like and must not be reported as though it were.** Submission 2's figures were measured on the hash-grouped split, where 45.9% of the test set had a near-duplicate in training. This submission's figure is measured on the clean split. The apparent 0.1458 macro-F1 gap therefore conflates two separate effects: the genuine cost of training from scratch, and the removal of leakage. Nothing in the present data separates them.

The experiment that would separate them

Re-running a single pretrained model — EfficientNetB0 fine-tuned — on the clean split would take approximately eleven minutes and would yield the two numbers needed:

	Contaminated split	**Clean split**
EfficientNetB0 fine-tuned	0.9973	*not yet measured*
Custom CNN from scratch	*not measured*	0.8515

Filling the top-right cell gives both the true cost of leakage for a pretrained model and a valid like-for-like transfer-learning comparison. It is the single most informative experiment still outstanding in this project.

5 Headroom for Submission 4

Quantity	Value
Baseline macro F1	0.8515
Test set size	666
Standard error at this accuracy	1.38 pp
Run-to-run variance (seed, shuffling, augmentation)	≈ 0.70 pp
Minimum detectable gain	**≈ 3.1 pp**
Headroom available	**14.8 pp**

The comparison is viable but not generously so. CBAM typically delivers 1–3 pp on tasks of this kind, which sits at the detection threshold. If the measured gain falls below 3 pp, the defensible responses are to run both models with three seeds — lowering the threshold to about 1.8 pp — or to report the result as inconclusive. Reporting a 1 pp gain from a single run as evidence that attention helps would not be supportable.

> **For contrast: had Submission 2's contaminated 0.9973 been used as the baseline, the gain required to clear noise would have exceeded the total headroom available, and Submission 4 could not have produced a result at all.** The leakage audit is what made this submission measurable.

6 The leakage audit

Conducted between Submissions 2 and 3, and the reason this submission uses a different split from its predecessor.

6.1 Motivation

Submission 2's best model reached 0.9972 accuracy — 719 of 721 test images correct — on a task characterised in Submission 1 as fine-grained. Submission 1 had controlled for duplication by grouping images with identical perceptual hashes, removing 608 redundant files (12.9%). That control proved incomplete.

6.2 Method

Every image was passed through EfficientNetB0 with ImageNet weights as a fixed feature extractor, producing a 1,280-dimensional vector. Vectors were L2-normalised and clustered by connected components over cosine similarity. The threshold was swept from 0.99 to 0.90.

The rationale: a perceptual hash is computed from greyscale brightness gradients, so any transformation that alters the channel mix alters the hash. A convolutional feature extractor trained with colour augmentation is largely invariant to exactly those transformations.

6.3 Findings

Threshold	Clusters	Redundant	%	Largest cluster
0.99	4,057	667	14.1	14
0.98	3,730	994	21.0	20
0.95	**2,777**	**1,947**	**41.2**	**73**
0.92	2,206	2,518	53.3	83
0.90	1,946	2,778	58.8	83

At the chosen threshold of 0.95, **zero** clusters span more than one class, indicating no cross-class over-merging.

6.4 What the augmentation actually was

The grids make the mechanism plain: the author's augmentation was **colour jitter**, not geometric transformation. This is precisely why a difference hash failed. Greyscale luminance is computed as $L = 0.299R + 0.587G + 0.114B$; a hue shift changes the channel mix, which changes L unevenly across the frame, which flips the gradient comparisons that constitute the hash — while leaving the photograph's content identical.

A difference hash is sensitive to exactly what the augmentation altered and blind to what it preserved. It was the worst available tool for this particular dataset.



Figure 4: The six largest clusters. Each row is one photograph reproduced under different colour transformations. The top row contains 73 copies of a single image. This is visual confirmation that the clustering is identifying genuine duplicates rather than merely similar photographs.

6.5 Consequence for Submission 2

Test images, Submission 2 split	721
With a near-duplicate in training	**331**
Contamination rate	**45.9%**
Genuinely held out	390 (54%)

6.6 An honest limit on this conclusion

Contamination is proven and large. It does **not**, on its own, explain Submission 2's accuracy. With only two errors in 721 predictions, accuracy on the 390 genuinely uncontaminated test images was between 0.9949 and 1.0000 whichever way the two errors fell. Something beyond duplicate leakage was also inflating that result — plausibly session correlation (photographs from the same match sharing stadium, kit and lighting), or a threshold that remains conservative, or a task that is simply easier than Submission 1 assumed.

The present submission's drop to 0.8514 on the clean split is consistent with the leakage having been substantial, but it cannot isolate the cause, because it also changed architecture and training regime. The eleven-minute experiment described in Section 5 would isolate it.

7 Conclusion

1. A 1.2 M-parameter network trained from random initialisation reaches **0.8515 macro F1** on a leakage-controlled split of this dataset.
2. It is limited by its training budget rather than by capacity or overfitting, and would improve with a longer schedule.
3. The per-class difficulty ordering is stable across every run in this project and is a property of the data.
4. The hardest confusion is `drive` ↔ `legglance-flick`, confirming Submission 1's visual prediction and correcting Submission 2's contaminated measurement.
5. Submission 4 has 14.8 pp of headroom against a 3.1 pp detection threshold. The comparison is viable, and would not have been without the audit.